

DWH

Принципы взаимодействия между СУБД



Проверить, идет ли запись

**Меня хорошо видно
&& слышно?**



Тема вебинара

Принципы взаимодействия между СУБД



Андрей Поляков

Senior Java/Groovy Developer/ Data Engineer

Об опыте:

- В отрасли бекенд разработки на java я более 6 лет.
- Занимался fullstack разработкой приложений, разработкой высоконагруженных compute-grid систем, а также микросервисов и etl-пайплайнов.
- Сейчас в роли старшего разработчика работаю над сервисами платежных систем в Unlimint.

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в telegram **#dwh-2024-12**



Задаем вопрос
в чат или ГОЛОСОМ



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом

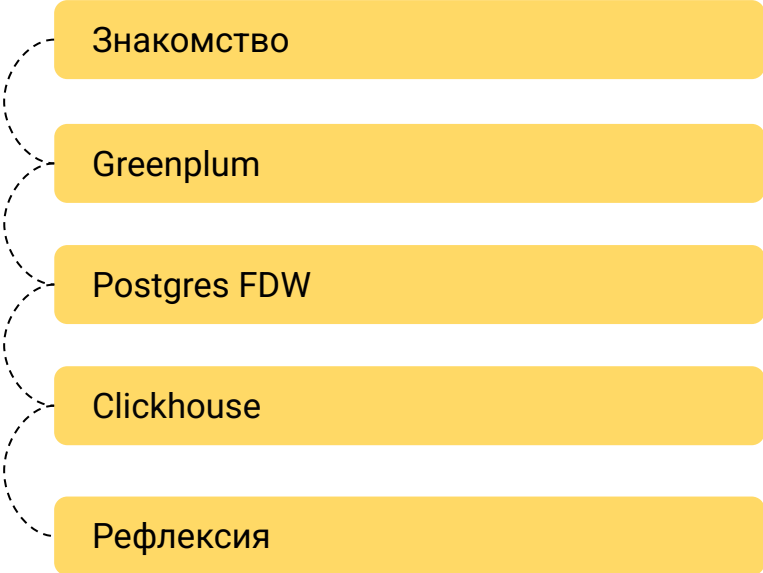


Документ



Ответьте себе или
задайте вопрос

Маршрут вебинара



Знакомство

Greenplum

Postgres FDW

Clickhouse

Рефлексия

Цели вебинара

К концу занятия вы сможете

1. Понять роль разных типов СУБД в DWH.
2. Изучить методы передачи и трансформации данных между системами.
3. Освоить инструменты для эффективного взаимодействия (pxf, external tables, fdw и пр.)

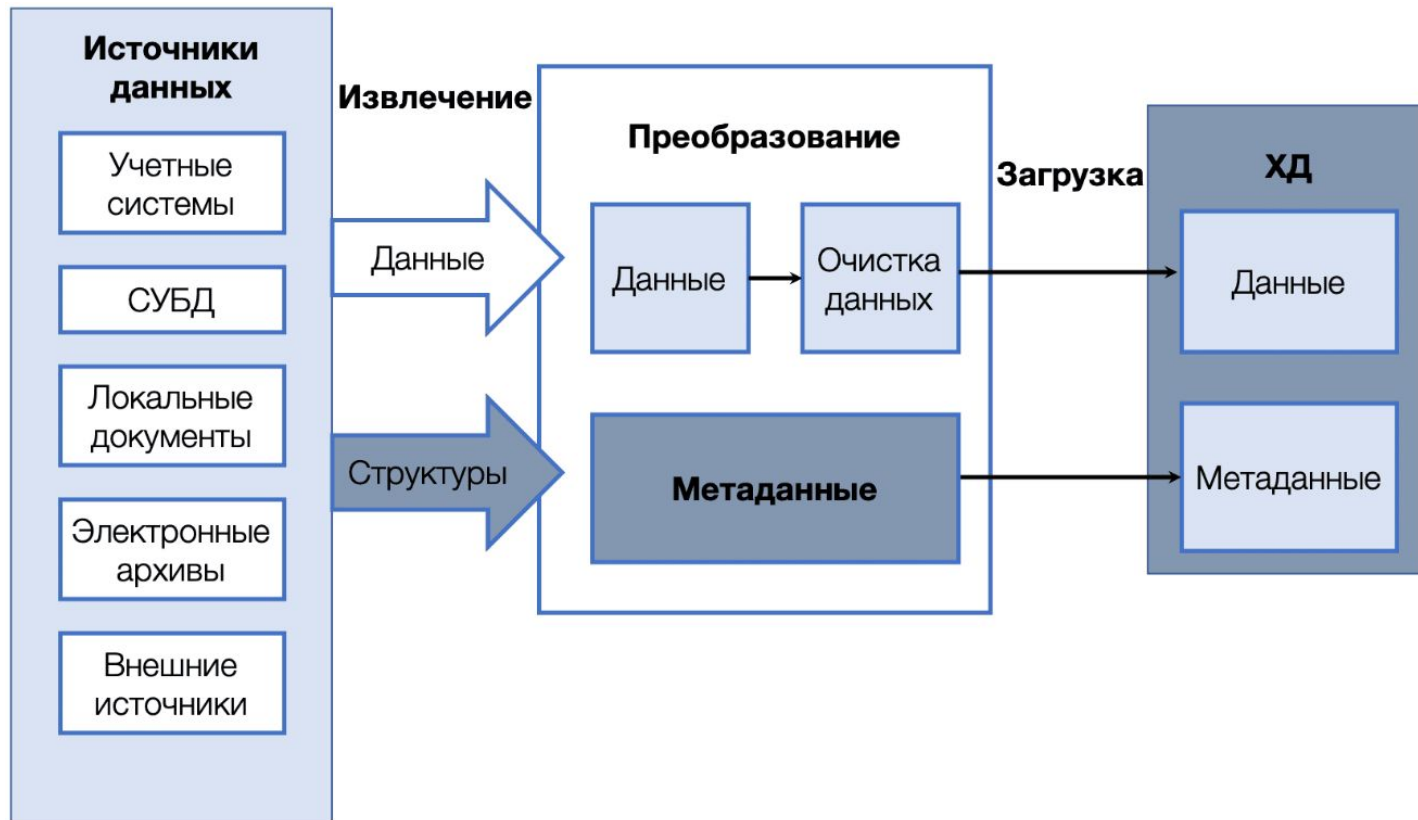
Смысл

Зачем вам это уметь

1. Закрепить навыки построения DWH as-code с помощью dbt
-

Подключение к внешним БД

Мотивация: ETL



Слои хранилища

- **Операционный слой первичных данных** (Primary Data Layer, Raw или Staging) — загрузка информации из систем-источников в исходном качестве и сохранением полной истории изменений.
- **Ядро хранилища** (Core Data Layer) — центральный компонент, консолидация данных из разных источников, приведение их к единым структурам и ключам.
- **Аналитические витрины** (Data Mart Layer) — данные преобразуются к структурам, удобным для анализа и использования в BI-дашбордах или других системах-потребителях.

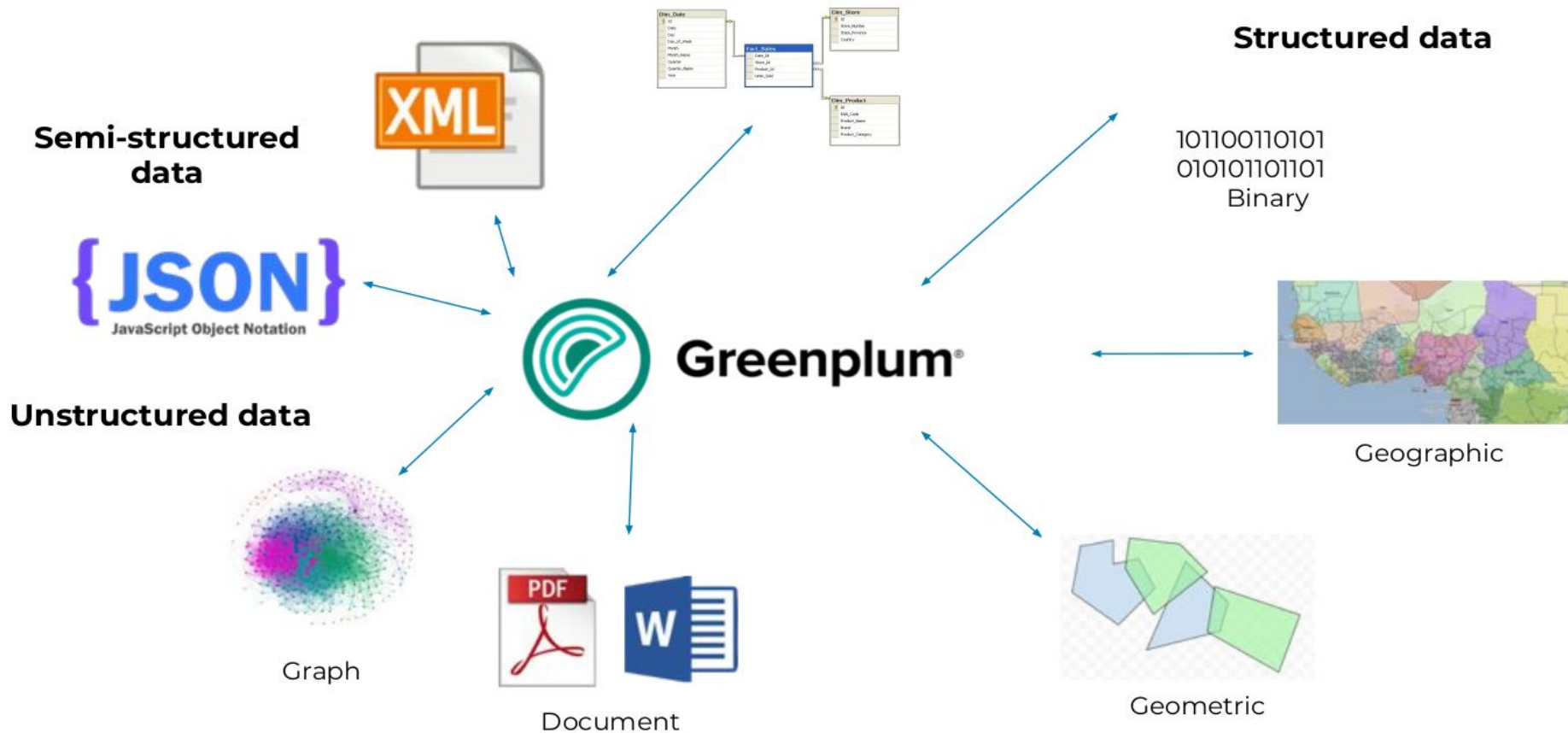
Инструменты ETL

- Airflow
- NiFi
- StreamSets
- ...

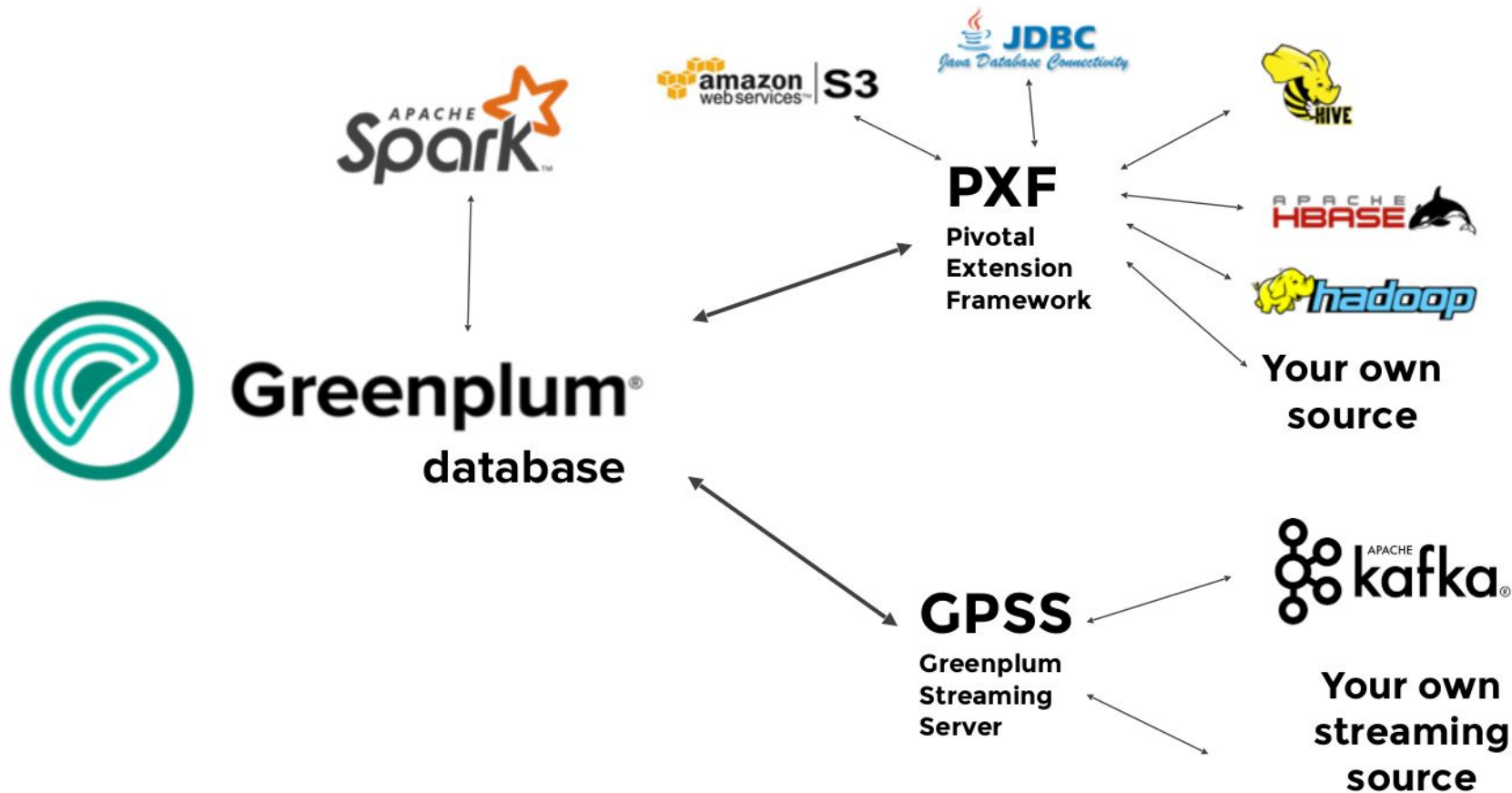
Можно ли обойтись без сторонних инструментов?

Greenplum

Типы и форматы данных



Источники данных



VMware Greenplum

- Data connectors:
 - Greenplum-NiFi Connector
 - Greenplum-Spark Connector
 - Greenplum-Informatica Connector
 - Greenplum-Kafka Integration
 - Greenplum Streaming Server
- gpcopy - utility for copying or migrating objects between Greenplum systems

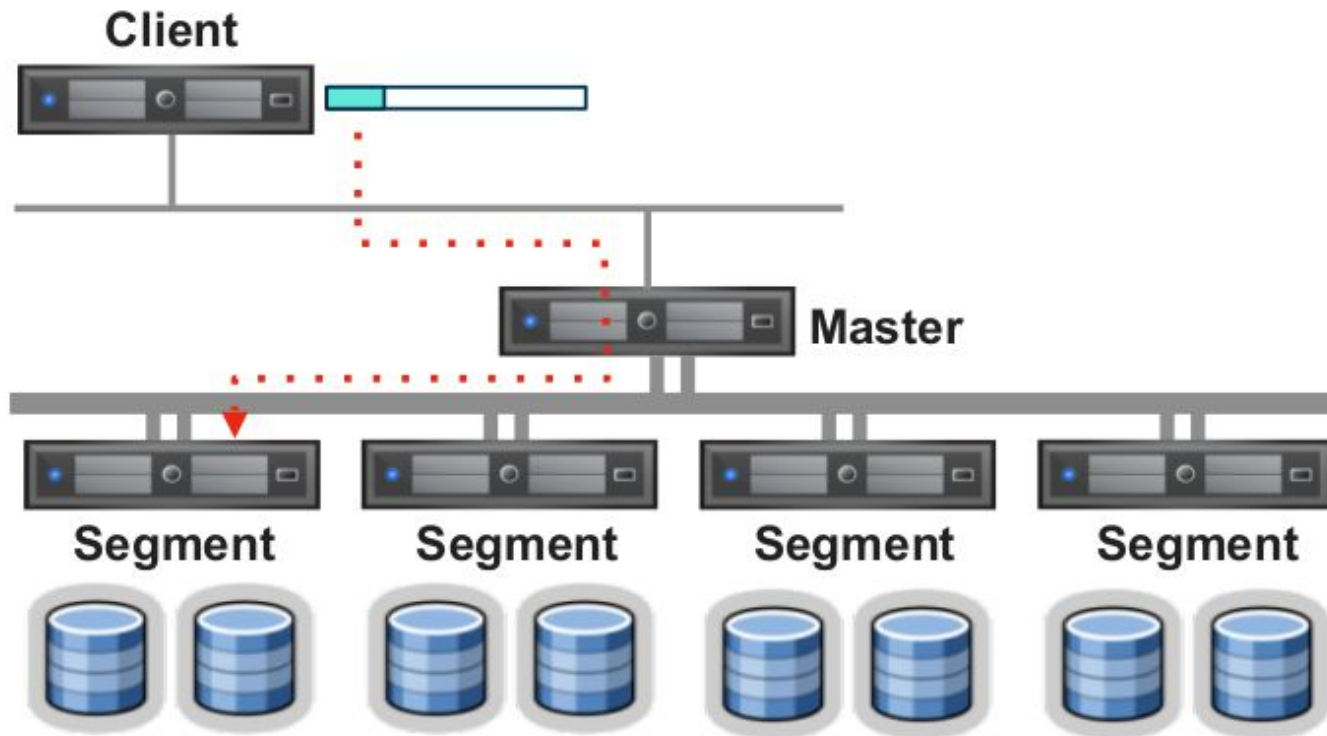
Загрузка данных

Method	Common Uses	Examples
INSERT	Operational Workloads OBDC/JDBC Interfaces	INSERT INTO performers (name, specialty) VALUES ('Sinatra', 'Singer');
COPY	Quick and easy data in Legacy PostgreSQL applications Output sample results from SQL statements	COPY performers FROM '/tmp/comedians.dat' WITH DELIMITER ' ';
External Tables & gpfdist protocol	High speed bulk loads Parallel loading using gpfdist protocol Local file, remote file, executable or HTTP based sources	INSERT INTO craps_bets SELECT g.bet_type, g.bet_dttm, g.bt_amt FROM x_allbets b JOIN games g ON (g.id = b.game_id) WHERE g.name = 'CRAPS';
GPLOAD	Simplifies external table method (YAML wrapper) Supports Insert, Merge & Update	gpload -f blackjack_bets.yaml
PXF	Access data from Hadoop, S3 and relational databases using JDBC	
GPSS	Streaming Engine	gpsscli submit --name json2 --gpss-port 5019 job4.yaml



SQL Insert Method

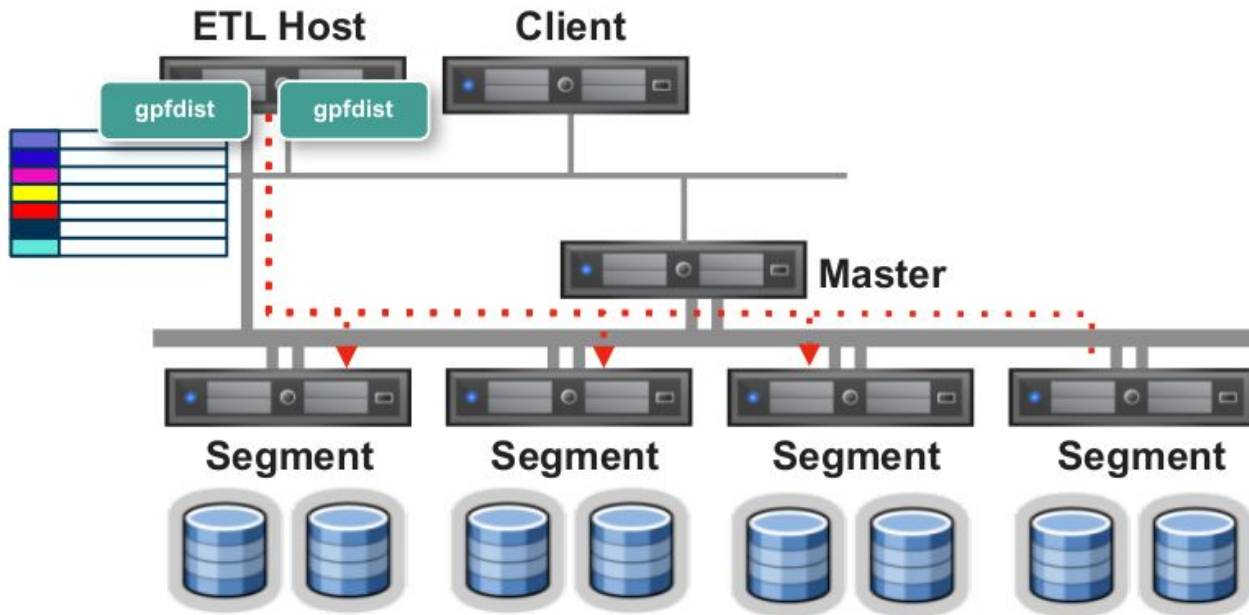
INSERT INTO <table> SELECT FROM <external table>



Методы параллельной загрузки

Provides full parallelism for best performance

- gpfdist : The parallel file distribution program
- gpload : Interfaces with gpfdist and external table
- gpss: Use gpfdist protocol and works with external tables

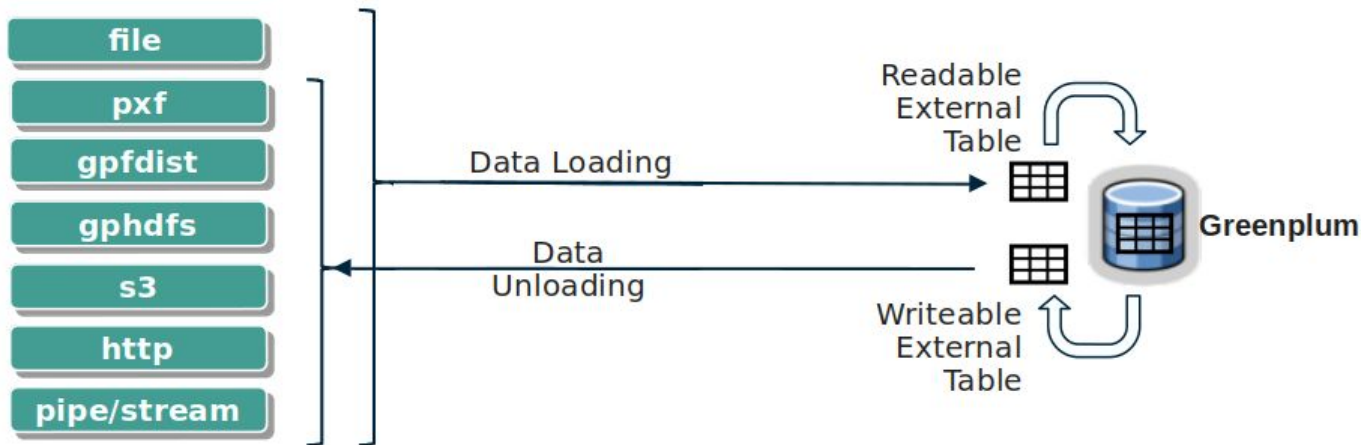


Внешние таблицы

- Позволяют SQL доступ к данным за пределами базы данных
- Могут быть доступны для чтения и записи
- Можно взаимодействовать с помощью операторов SELECT и INSERT
- Обычно используется для ETL и загрузки/выгрузки данных
- Преимущества:
 - Параллелизм (данные не проходят через главный хост)
 - Используется с инструментами ETL
 - Отслеживание плохих записей
 - Несколько источников данных

Типы внешних таблиц

Type	Usage	Protocol
Readable External Tables	Read a file outside the DB (possibly outside the cluster)	file, gpfdist, gphdfs, pxf, s3
Readable External Web Tables	URL or execution of a command (bat, shell, ...) The result is readable with a SELECT command	http, EXECUTE
Writeable External Tables	Write a file outside the DB (possibly outside cluster)	gpfdist, gphdfs, pxf, s3
Writeable External Web Tables	Execution of a command (bat, shell, ...) which retrieves through a pipe data from a SELECT query	EXECUTE



Внешние таблицы из файлов

- Укажите в LOCATION столько URI, сколько сегментов у вас есть
- Каждый URI указывает на внешний файл данных или источник данных
- URI не обязательно должны существовать до определения внешней таблицы
- URI должен существовать при запросе данных

```
CREATE EXTERNAL TABLE ext_expenses (  
    name text,  
    date date,  
    amount float4,  
    category text,  
    description text )  
  
LOCATION (  
    'file://seghost1/dbfast/external/expenses1.csv',  
    'file://seghost1/dbfast/external/expenses2.csv',  
    'file://seghost2/dbfast/external/expenses3.csv',  
    'file://seghost2/dbfast/external/expenses4.csv' )  
  
FORMAT 'CSV' ( HEADER );
```

You can define as many URIs as you have segments

If using the file:// protocol, the external data file or files must reside on a segment host in a location accessible by the Greenplum super user.

CSV format with header

Внешние таблицы из gpfdist

- Каждый сегмент открывает соединение с удаленным gpfdist agent
- Каждый сегмент запрашивает блок данных (ie. just a bunch of data only parsed for EOL)
- После того, как данные попадут на сегмент, DISTRIBUTED BY вызывается, которое, скорее всего, приведет к перемещению данных в другой сегмент.

```
gpfdist -d /var/load_files/expenses1 -p 8081 >> gpfdist.log 2>&1 &  
gpfdist -d /var/load_files/expenses2 -p 8082 >> gpfdist.log 2>&1 &
```

```
CREATE EXTERNAL TABLE ext_expenses (  
    name text,  
    date date,  
    amount float4,  
    description text)  
LOCATION (  
    'gpfdist://etlhost:8081/*',  
    'gpfdist://etlhost:8082/*')  
FORMAT 'TEXT' (DELIMITER '|')  
ENCODING 'UTF-8'  
LOG ERRORS  
SEGMENT REJECT LIMIT 10000 ROWS ;
```

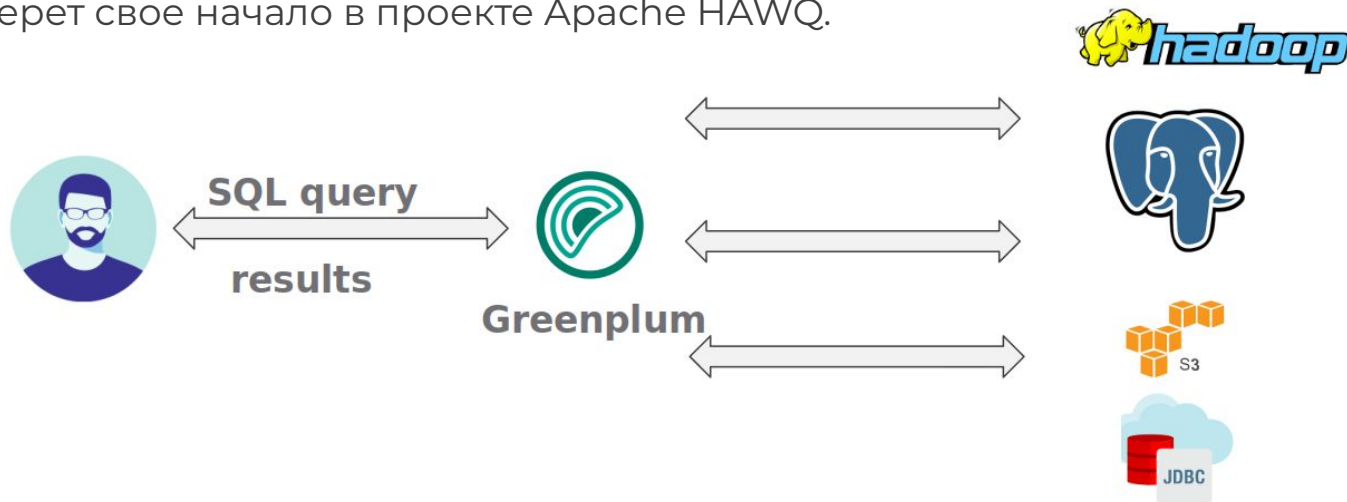
Multiple gpfdist

Error Logging

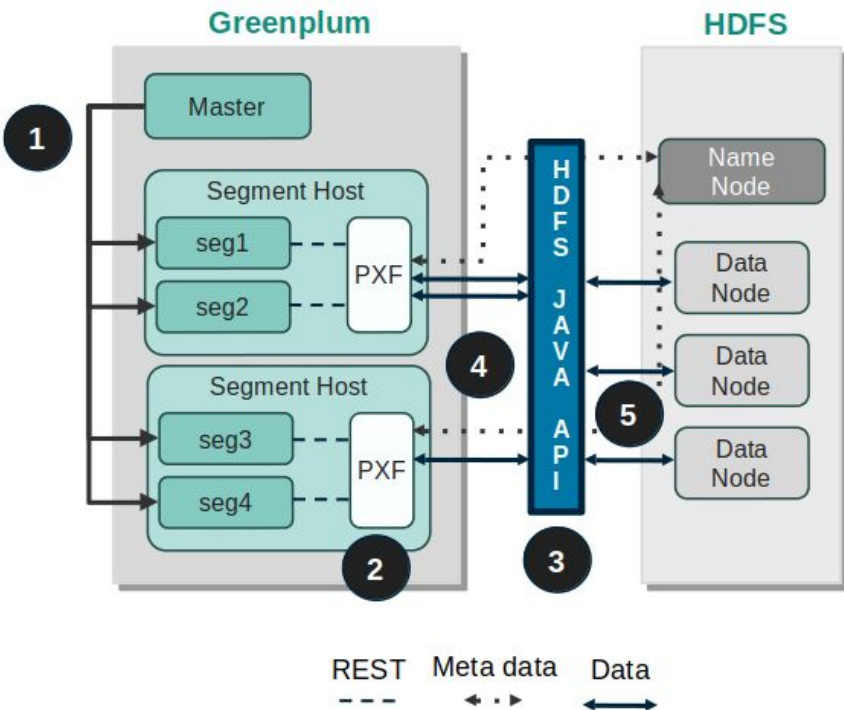
```
INSERT INTO expenses (SELECT * FROM ext_expenses);
```

PXF Protocol

- The Greenplum **Platform Extension Framework (PXF)** обеспечивает параллельный, высокопроизводительный доступ к данным и федеративные запросы по разнородным источникам данных через встроенные коннекторы, которые сопоставляют определение внешней таблицы базы данных Greenplum с внешним источником данных.
- Позволяет GPDB получать доступ к внешним источникам данных.
- Простота использования и отсутствие необходимости в ETL
- PXF берет свое начало в проекте Apache HAWQ.

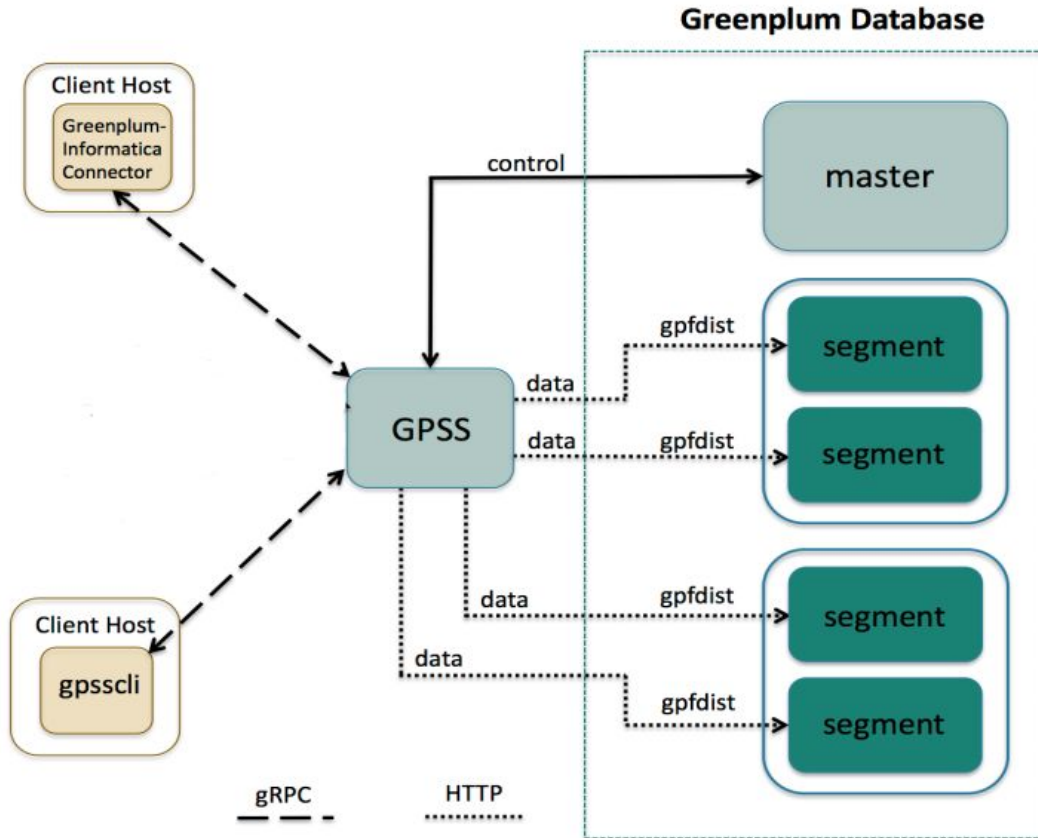


PXF Architecture



- 1 Master node отправляет запрос на все segment host
- 2 PXF agent создает поток для segment instance
- 3 Вызывается HDFS Java API to request metadata information for the HDFS file from the HDFS NameNode
- 4 Provides the metadata information returned by the HDFS NameNode to the segment instance.
- 5 The segment instance отправляет запрос PXF agent-у для чтения назначенных данных.
The PXF agent вызывает HDFS Java API чтобы считать данные и отправить их на segment instance.

GPSS (GreenPlum Streaming Server)



Postgres FDW

Postgres FDW

Стандарт **SQL/MED 2003** (Management of External Data):

- **FDW**: Средства для чтения данных как набора реляционных таблиц под управлением одного или нескольких внешних источников (SQL-интерфейс для доступа к не SQL данным, таким, как файлы или, например, список писем в почтовом ящике)
- **Datalink** позволяет управлять удаленным SQL-сервером

Какие есть FDW?

- PostgreSQL
- Oracle, SQL
- Server
- MySQL
- Cassandra
- Redis
- RethinkDB
- Ldap
- файлы типа CSV, JSON, XML

Пример: FDW для mysql

[GitHub - EnterpriseDB/mysql_fdw: PostgreSQL foreign data wrapper for MySQL](https://github.com/EnterpriseDB/mysql_fdw)

Установка:

```
git clone https://github.com/EnterpriseDB/mysql_fdw.git  
PATH=$PATH:/usr/pgsql-9.5/bin USE_PGXS=1 make install
```

Установка расширения:

```
CREATE EXTENSION mysql_fdw;
```

Запуск сервера FDW:

```
CREATE SERVER mysql_server_data FOREIGN DATA WRAPPER mysql_fdw  
    OPTIONS (host '127.0.0.1', port '3306');  
GRANT USAGE ON FOREIGN SERVER mysql_server TO user;
```

Mapping пользователей:

```
CREATE USER MAPPING FOR user SERVER mysql_server_data  
    OPTIONS (username 'data', password 'datapass');
```



Подключить таблицу MySQL в PostgreSQL

```
CREATE FOREIGN TABLE orders_2024 (  
    id int,  
    customer_id int,  
    order_date timestamp)  
SERVER mysql_server_data  
OPTIONS (dbname 'data', table_name 'orders');
```

```
CREATE FOREIGN TABLE warehouse (  
    id int,  
    name text,  
    created timestamp)  
SERVER mysql_server_data  
OPTIONS (dbname 'data', table_name 'warehouse');
```

Взаимодействие с внешней таблицей

```
INSERT INTO warehouse values (1, 'UPS', current_date);  
INSERT INTO warehouse values (2, 'TV', current_date);  
INSERT INTO warehouse values (3, 'Table', current_date);
```

```
SELECT * FROM warehouse ORDER BY 1;
```

id	name	created
1	UPS	10-JUL-20 00:00:00
2	TV	10-JUL-20 00:00:00
3	Table	10-JUL-20 00:00:00

```
DELETE FROM warehouse where id = 3;
```

```
UPDATE warehouse set name = 'UPS_NEW' where id = 1;
```

Анализ быстродействия

```
EXPLAIN VERBOSE SELECT warehouse_id, warehouse_name
FROM warehouse
WHERE warehouse_name LIKE 'TV'
limit 1;
```

QUERY PLAN

```
-----
Limit (cost=10.00..11.00 rows=1 width=36)
  Output: warehouse_id, warehouse_name
    -> Foreign Scan on public.warehouse (cost=10.00..1010.00 rows=1000 width=36)
      Output: warehouse_id, warehouse_name
      Local server startup cost: 10
      Remote query: SELECT `id`, `name` FROM `db`.`warehouse` WHERE
        ((`warehouse_name` LIKE BINARY 'TV'))
```


Import a MySQL database as schema to PostgreSQL

- Команда создаёт на локальном сервере определения сторонних таблиц, соответствующие таблицам или представлениям, существующим на удалённом сервере.
- Но! Если импортируемые удалённые таблицы содержат столбцы пользовательских типов данных, на локальном сервере должны быть совместимые типы с теми же именами.

```
IMPORT FOREIGN SCHEMA someschema  
FROM SERVER mysql_server  
INTO public;
```

Updatable

- По умолчанию все внешние таблицы допускают обновление
- Можно сделать read-only с помощью параметра updatable (но! на удаленном сервере не проверяется).

Соединения

- Соединение устанавливается при первом запросе, в котором участвует внешняя таблица, связанная со сторонним сервером.
- Соединение сохраняется и повторно используется для последующих запросов в том же сеансе.
- Для разных пользователей отдельное соединение устанавливается для каждого сопоставления пользователей.

Транзакции

- Для удаленной транзакции выбирается режим изоляции **SERIALIZABLE**, когда локальная транзакция открыта в режиме SERIALIZABLE; в противном случае применяется режим **REPEATABLE READ**.
- 2PC не поддерживается
- postgres_fdw открывает транзакцию на удалённом сервере, если такая транзакция еще не была открыта для текущей локальной транзакции.
- Удалённая транзакция фиксируется или прерывается, когда фиксируется или прерывается локальная транзакция.

Полезные команды в psql

- **\des** - list foreign servers
- **\deu** - list uses mappings
- **\det** - list foreign tables
- **\dtE** - list both local and foreign tables
- **\d <name of foreign table>** - show columns, data types, and other table metadata

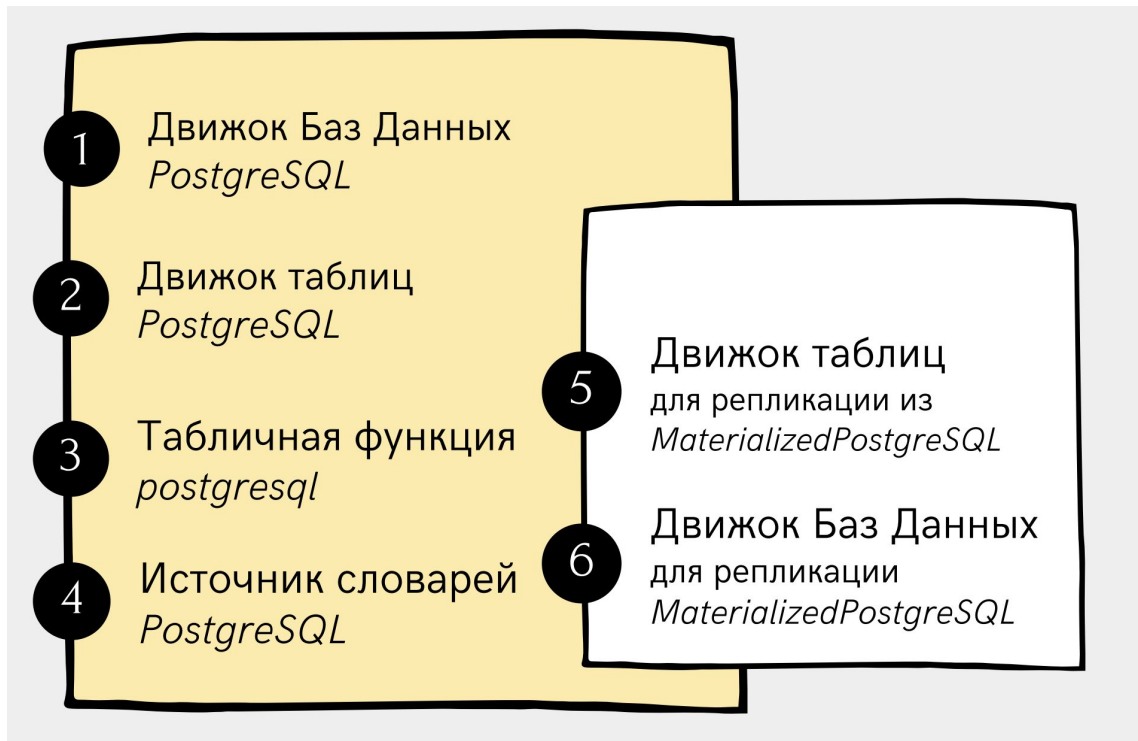
Другие FDW

- <https://github.com/rankactive/cassandra-fdw>
- https://github.com/pgspider/mongo_fdw
- https://github.com/pgspider/parquet_s3_fdw

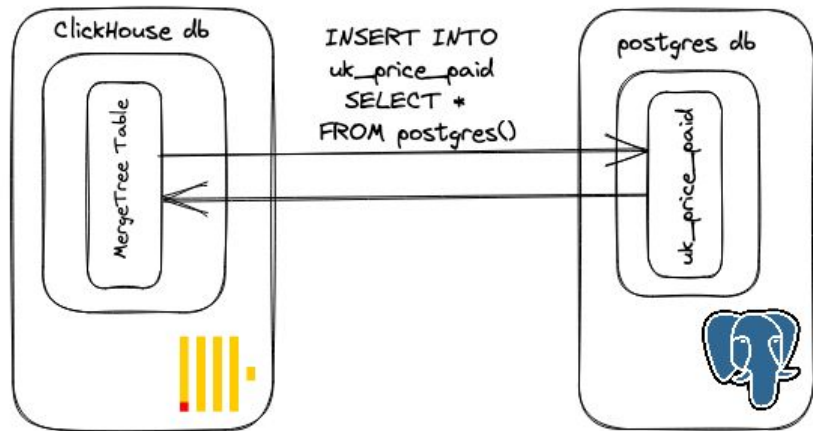
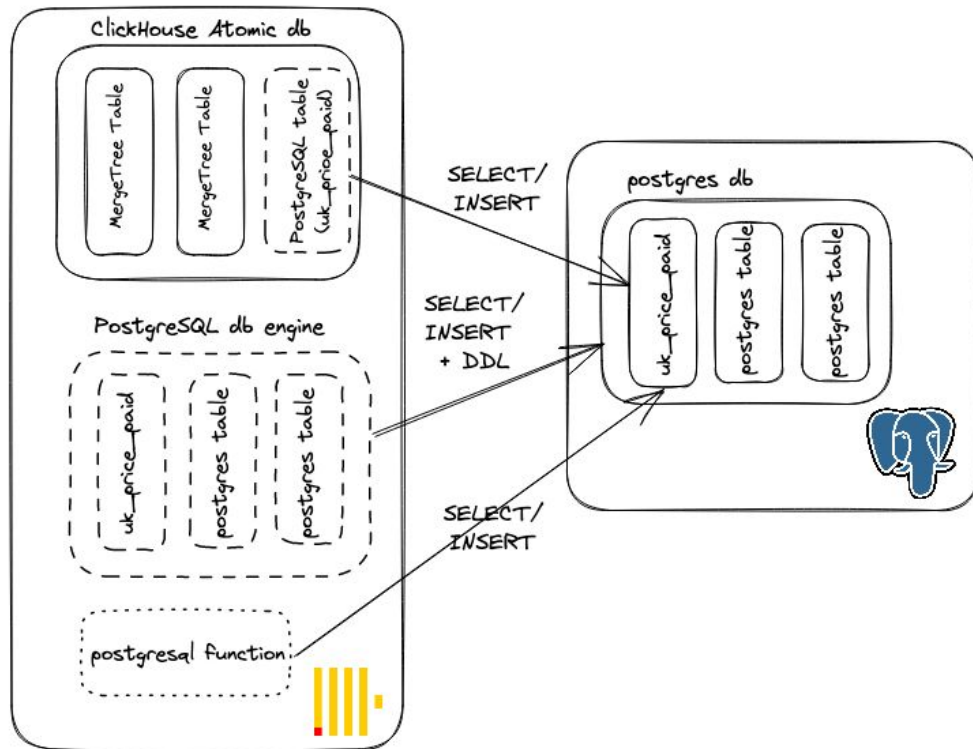
Multicorn - это обобщенный Foreign Data Wrapper (FDW, интерфейс для подключения внешних источников данных, устоявшегося русского названия пока нет), предоставляющий возможность разработки конкретных FDW на языке Python, что упрощает их разработку.

ClickHouse

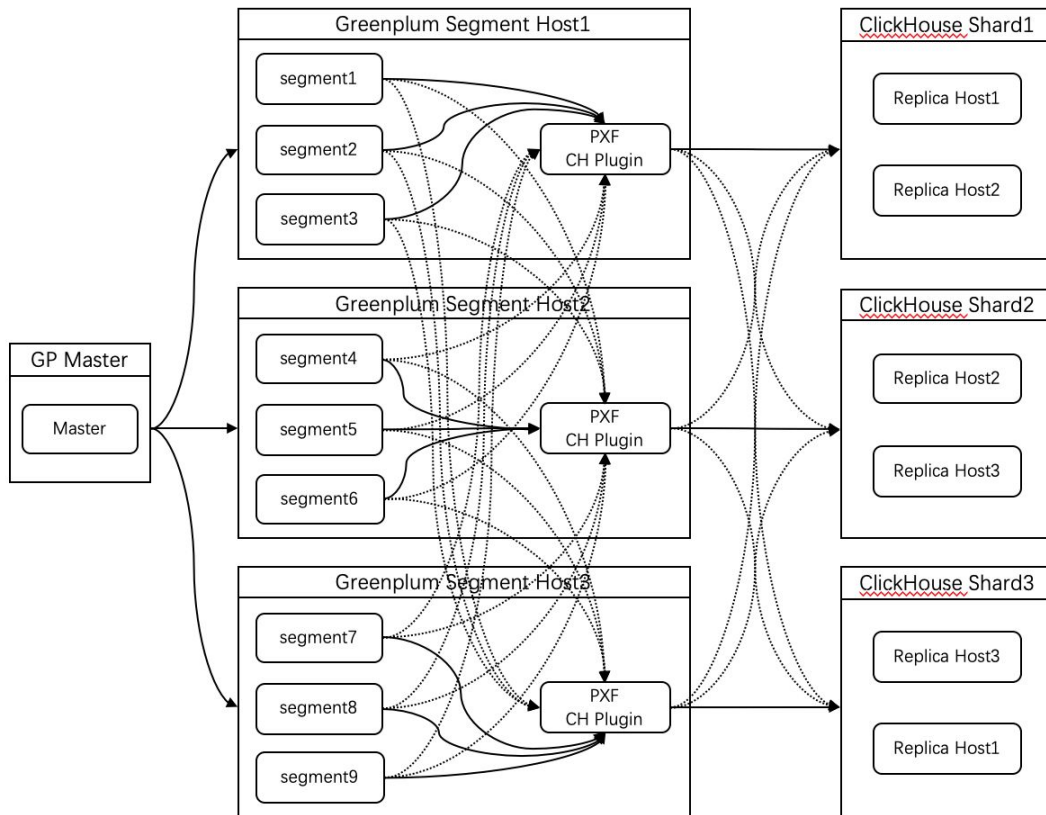
Интеграции ClickHouse и Postgres



Интеграции ClickHouse и Postgres



Интеграции ClickHouse и Greenplum



Интеграции ClickHouse и Greenplum

Creating an external table using an SQL query

The SQL query syntax to create an external table is as follows:

```
CREATE [WRITABLE] EXTERNAL TABLE <table_name>
(<column_name> <data_type> [, ...])
LOCATION('pxf://<data_path_or_table_name>?PROFILE=<profile_name>&SERVER=<source_name>')
FORMAT '[TEXT|CSV|CUSTOM]';
```

Where:

- `<table_name>` : Name of the external table to be created in the Greenplum® cluster.
- `<column_name>` : Column name.
- `<data_type>` : Column data type. It must match the column data type in the external DBMS table.
- `<data path or table name>` : External object name, see [examples of external tables](#).
- `PROFILE` : Standard interface to an external DBMS (profile), e.g., `JDBC`. The list of possible values depends on the connection type:
 - [S3](#)
 - [JDBC](#)
 - [HDFS and Hive](#)
- `SERVER` : Name of the external PXF data source.

Движки

- table engine PostgreSQL
 - данные хранятся в Postgres
 - в зависимости от настроек метаданные хранятся либо на клике, либо узнаются во время запроса
- database engine PostgreSQL
 - view в реальном времени на все таблицы базы в PG

Движки Materialized

- table engine MaterializedPostgreSQL
 - данные хранятся в ClickHouse
 - метаданные тоже в ClickHouse
- database engine MaterializedPostgreSQL
 - данные уже на диске в клике

Experimental

Примеры

```
clickhouse :) CREATE TABLE postgres_table
              ENGINE = PostgreSQL('postgres1:5432', 'postgres_database', 'postgres_table',
                                   'user', 'pwd')

clickhouse :) CREATE DICTIONARY postgres_dict (id UInt32, val UInt32)
              PRIMARY KEY id
              SOURCE(PGSQL(
                port 5432
                host 'localhost'
                user  'postgres_user'
                password 'pwd'
                db 'postgres_database'
                table 'postgresql_table'))
              LIFETIME(MIN 1 MAX 2)
              LAYOUT(HASHED());
```

Примеры

```
clickhouse :) CREATE DATABASE postgres_database_database
              ENGINE = PostgreSQL('postgres1:5432', 'postgres_database', 'postgres_table',
                                   'user', 'pwd')
```

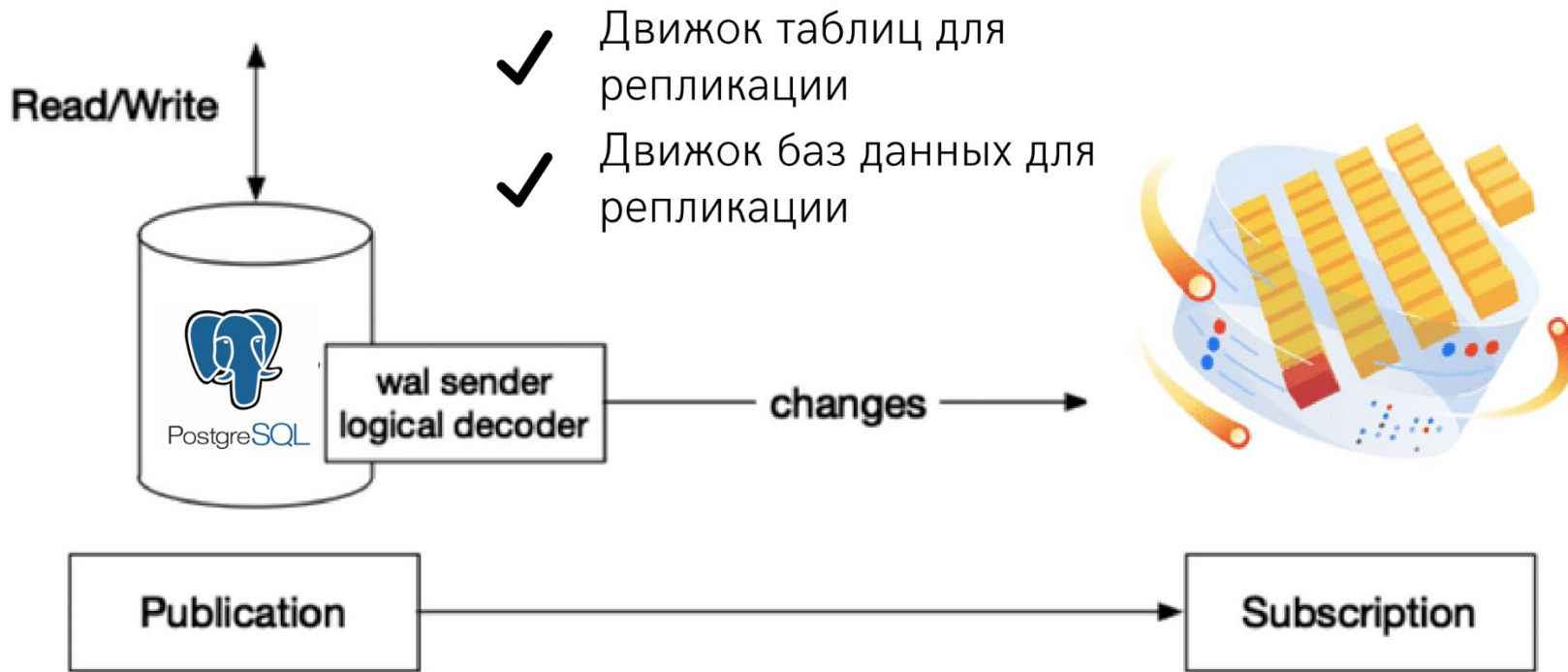
```
clickhouse :) SELECT * FROM postgres_database.table1;
```

```
clickhouse :) INSERT INTO postgres_database.table1;
```

```
clickhouse :) SELECT joined FROM postgres_database.table1 ANY LEFT JOIN (SELECT number * 2
AS number, number * 10 + 1 AS joined FROM system.numbers LIMIT 10) js2 USING number LIMIT 10
```

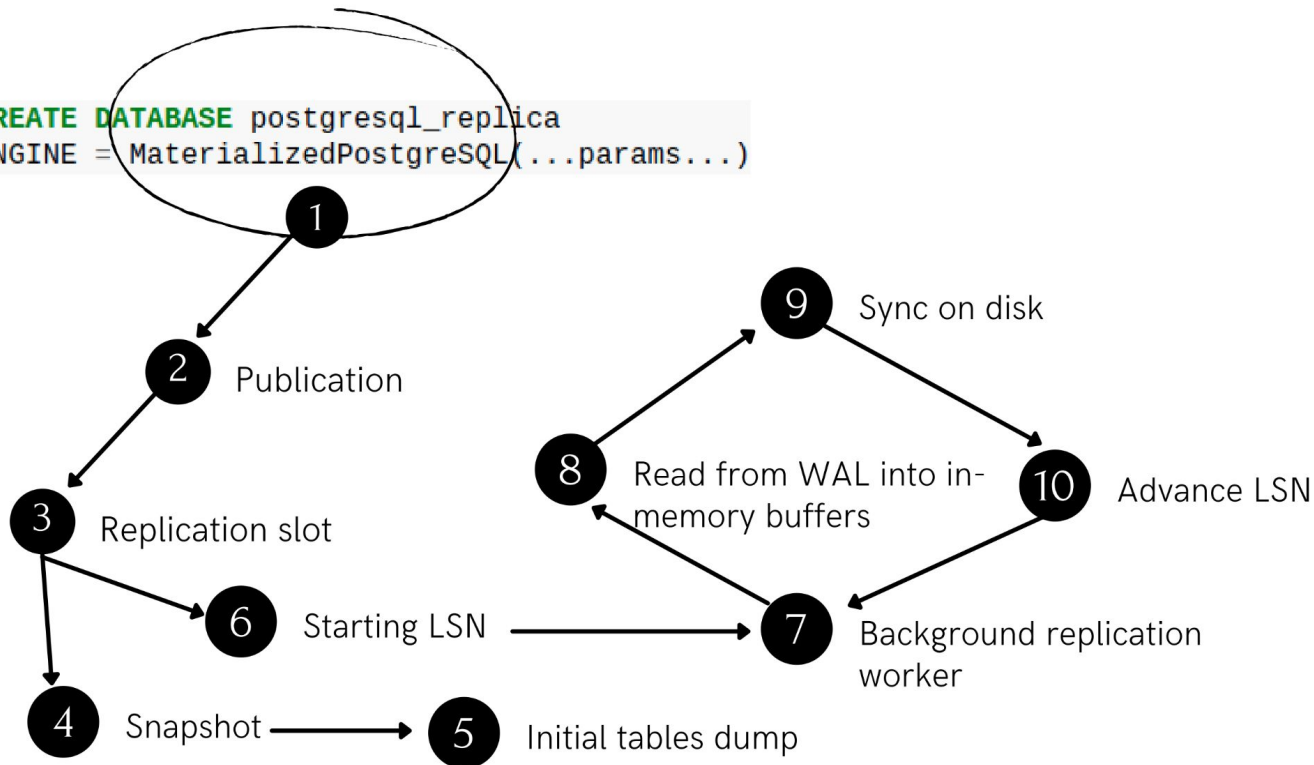
```
clickhouse :) INSERT INTO FUNCTION postgresql('localhost:5432', 'postgres_database',
        'postgres_table', 'user', 'pwd') SELECT number FROM numbers(10)
```

Репликация данных из PG в СН



Репликация данных из PG в CH

```
CREATE DATABASE postgresql_replica  
ENGINE = MaterializedPostgreSQL(...params...)
```



Рефлексия

Рефлексия



С какими впечатлениями уходите с вебинара?



Как будете применять на практике то, что узнали на вебинаре?

Следующий вебинар



19 февраля 2023

DBT: Extending with modules



Ссылка на вебинар
будет в ЛК за 15 минут



Материалы
к занятию в ЛК —
можно изучать



Обязательный материал
обозначен красной
лентой



**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**

<https://otus.ru/polls/56425/>

Спасибо за внимание!

Приходите на следующие вебинары



Поляков Андрей

Senior Java/Groovy Developer at Unlimint

